

1. Memory Allocation: Static vs. Dynamic

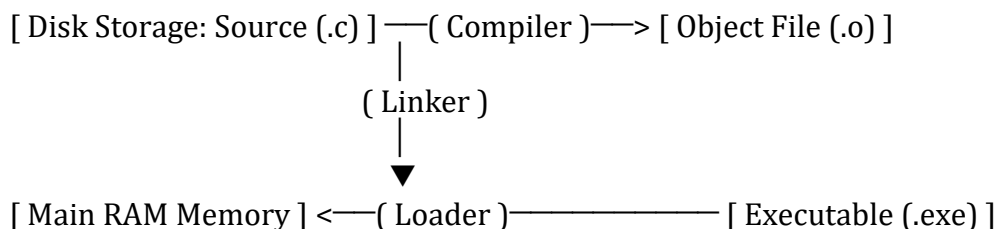
High-Level Comparison

Memory allocation strategies determine how and when a C program requests workspace blocks from the underlying Operating System.

Feature	Static Memory Allocation (SMA)	Dynamic Memory Allocation (DMA)
Allocation Mechanism	Instantiated via normal variable declarations (e.g., <code>int x;</code>).	Provisioned explicitly by calling specialized library functions.
Allocation Timing	Fixed at compile time.	Resolved dynamically during system execution time.
Flexibility	Non-flexible; cannot be increased or decreased at runtime.	Highly flexible; adapts dynamically to application workloads.
Risks	Memory wastage (oversizing) or rigid sizing bounds.	Memory leaks if not managed manually by the programmer.
Primary Functions	N/A	<code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> , <code>free()</code> .

2. Memory Execution Pipeline & Loader

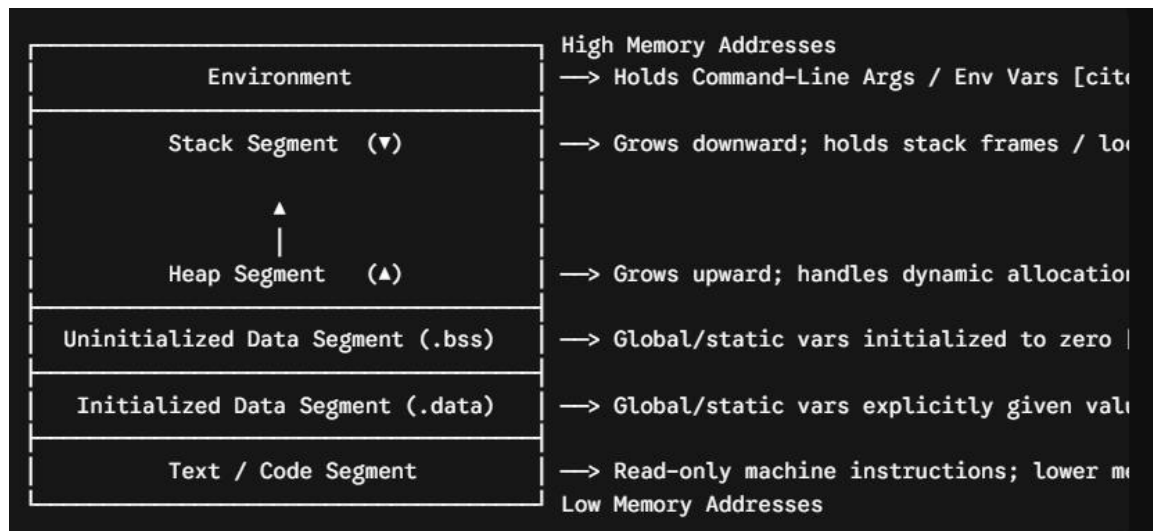
When you write code, it flows through a sequential translation pipeline before occupying physical RAM:



Loader's Role: The **Loader** is the operating system component responsible for taking the compiled executable (.exe) from persistent disk storage (HDD/SSD) and mapping it into main memory (RAM) as an active **Process**.

3. Memory Segments of a C Program

An executing process splits its assigned RAM layout into specific functional zones:



Deep-Dive into Segments

Text/Code Segment: Stores raw machine code instructions. It is **read-only** to protect against runtime modifications and sits at lower addresses to prevent accidental overwrites.

Initialized Data Segment: Holds global and static variables explicitly initialized by the developer. These persist throughout the program's lifecycle.

Uninitialized Data Segment (BSS): Holds global and static variables that are declared but not explicitly initialized. The system automatically sets them to zero at startup.

Memory Segment Code Trace Example:

```
int val; -----> Uninitialized Data Segment (.bss)
const char *str = "HW"; -----> Initialized Data Segment (.data)
int main()
{
    static int a = 10; -----> Initialized Data Segment (.data)
```

```
static int b;    -----> Uninitialized Data Segment (.bss)
return 0;
}
```

Stack Segment: Allocates isolated **Stack Frames** (or activation records) whenever a function is called. These store execution parameters and local variable targets. The frame is automatically popped and wiped as soon as the function hits a return statement.

Heap Segment: An unorganized pool of shared system memory where raw allocations occur at runtime via explicit manual pointer functions.

4. C Dynamic Sizing Functions (<stdlib.h>)

A. malloc() (Memory Allocation)

Behavior: Allocates a single contiguous block of raw memory of a requested byte size. It leaves this block filled with uninitialized **garbage values**.

Return Type: Returns a generic void* pointing to the base address of the block. Because malloc does not know what type of data you intend to store, you should explicitly type-cast its return value.

Recommended syntax pattern: `int *ptr = (int *)malloc(n * sizeof(int));`

💡 **Best Practice:** Always use `sizeof(type)` instead of hardcoding numeric byte counts, since primitive sizes vary across system architectures.

B. calloc() (Contiguous Allocation)

Behavior: Allocates **multiple sequential blocks** of memory of identical size.

Key Differences from malloc():

It takes two arguments: the number of elements and the byte size per element.

It automatically initializes all allocated bytes to **zero (0)**.

`int *ptr = (int *)calloc(5, sizeof(int));` ----> **Allocates 5 blocks of 4-byte integers (20 bytes total)**

Defensive Guard: Checking for Allocation Failure

If the system runs out of memory, both `malloc()` and `calloc()` will return `NULL`.

Always verify your pointer before trying to use it:

```
if (ptr == NULL)
{
    printf("Sufficient memory not available\n");
    exit(1);
}
```

C. `realloc()` (Re-allocation)

Behavior: Resizes an existing heap block (expanding or shrinking it) without losing its current data.

Under-the-Hood Mechanics:

Case 1 (In-Place Extension): If adjacent space is available in the heap, `realloc` extends the current block inline and keeps the same base address.

Case 2 (Move Allocation): If adjacent space is blocked, it allocates a completely new block elsewhere, copies the existing data over, automatically frees the old block, and returns the new address.

Failure: If the reallocation fails, it returns `NULL`, leaving the original memory block intact and un-deallocated.

```
int *ptr = (int *)malloc(5 * sizeof(int));
int *new_ptr = (int *)realloc(ptr, 7 * sizeof(int)); ---> Expands the allocation to
hold 7 integers
if (new_ptr != NULL)
{
    ptr = new_ptr; -----> Update pointer securely
}
```

D. `free()` (Memory Deallocation)

Behavior: Releases dynamically allocated memory back to the heap pool so it can be reused by the system.

Crucial Rule: Heap allocations do not clean themselves up automatically; they persist until the program finishes executing unless explicitly freed.

`free(ptr);` ----> Releases the block [cite: 950]
`ptr = NULL;` -----> Prevents it from becoming a Dangling Pointer

⚠ **Dangling Pointer Warning:** Calling `free(ptr)` releases the memory block but leaves the pointer variable holding the old address. Trying to access it afterward causes undefined behavior. Always set the pointer to `NULL` immediately after freeing it.

5. Memory Leaks in C

What is a Memory Leak?

A **Memory Leak** occurs when a program allocates memory on the heap but loses its tracking pointer without freeing the memory first. This leaves the allocated blocks trapped and inaccessible to the rest of the system.

Visualizing a Loop-Driven Memory Leak

```
int main()
{
    int ch = 1;
    int *ptr;
    while (ch < 50)
    {
        ptr = (int *)malloc(4000 * sizeof(int)); ----> Allocates ~16KB every iteration without a matching free()
        ----> ptr is overwritten in the next iteration, losing the address to the previous block forever!
        ch++;
    }
    return 0;
}
```

Cumulative Heap Corruption Diagram

Iteration 1: [ptr] —> [Block A (16KB)] (Orphaned!)
Iteration 2: [ptr] —> [Block B (16KB)] (Orphaned!)
Iteration 3: [ptr] —> [Block C (16KB)] (Active pointer target)

Consequences

Over time, memory leaks drain the pool of available system memory, heavily degrading computer performance.

Eventually, the operating system slows down significantly or crashes due to memory exhaustion.

Note:

For making your understanding more clear and simple, you must visit the handwritten note provided in the same folder **15-Memory_Allocation**. It features hand-drawn execution traces, explicit pointer type-casting callouts, and raw compiler-specific memory padding rules.